SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	Tharun
Register Number	1924260280
Course Code & Name	C Programming
Slot	B
Assignment / Task Title	Q1 – District Blood Bank Network (Tasks A1–A5, B1–B4)
GitHub Repository URL	N/A
Submission Date	12 November 2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK A1 — Decompose the Program into Functions (7 Marks)

Task / Problem Statement

Case study: a district blood bank registers donors, finds eligible donors of a given blood group, updates stock after every donation and issue, and keeps all data in files. A1.1: break the program into functions and draw a structure chart. A1.2: write the prototype of every function and decide, for each parameter, between pass-by-value, pass-by-pointer and const pointer. A1.3: return two values (number of eligible donors and ID of the youngest eligible donor) in two different ways and compare them. Alternative: analyse what changes if global variables are used instead of parameters.

Aim

To split the blood bank program into small single-purpose functions with clean, justified interfaces, and to compare the ways a C function can return more than one value.

Concept / Theory

A function hides one job behind a name and an interface. In C every argument is passed by value, so a function receives a copy; to let a function change the caller's data, or to avoid copying a big structure, the address is passed (a pointer). A pointer to data that the function only reads is declared const, which lets the compiler reject accidental writes. C can return only one value, so a second result is returned either through an output pointer parameter or by packing both values in a struct. A structure chart shows which function calls which. Good decomposition gives high cohesion (one job per function) and low coupling (functions talk only through parameters).

Procedure / Algorithm

1. List the jobs of the program: register, validate group, check eligibility, find donors, update stock, report, load and save files.
2. Give each job one function and draw the structure chart with main() at the top (A1.1).
3. Write the prototype of each function and decide value / pointer / const pointer for every parameter (A1.2).
4. Design the 'count + youngest ID' function twice: with output pointer parameters and with a returned structure (A1.3).
5. Implement both versions in C, run them on the same donor list and confirm identical results.
6. Compare the designs and analyse the global-variable alternative (coupling, testing, reuse, time, memory).

Program / Source Code

Pseudocode

```
ALGORITHM eligible_ptr(list, n, group, OUT count, OUT youngest_id)          // design 1
BEGIN
   count <- 0 ; youngest_id <- -1 ; best_age <- infinity
   FOR i <- 0 TO n-1 DO
      IF list[i].group = group AND is_eligible(list[i]) THEN
         count <- count + 1
         IF list[i].age < best_age THEN best_age <- list[i].age ; youngest_id <- list[i].id
END

ALGORITHM eligible_struct(list, n, group) RETURNS EligibleResult             // design 2
BEGIN
   r.count <- 0 ; r.youngest_id <- -1 ; best_age <- infinity
   (same loop as above, writing into r)
   RETURN r
END

ALGORITHM is_eligible(d) RETURNS 1 or 0      -- age between 18 and 65 AND at least 90 days since last donation
```

A1 — C program (both designs)

```c
#include <stdio.h>
#include <string.h>

typedef struct { int id; char name[24]; char group[4]; int age; int days_since; } Donor;
typedef struct { int count; int youngest_id; } EligibleResult;

/* eligible: age 18-65 and at least 90 days since the last donation (by value read-only copy not needed: const pointer) */
static int is_eligible(const Donor *d)
{
    return d->age >= 18 && d->age <= 65 && d->days_since >= 90;
}

/* Design 1: results returned through pointer (output) parameters */
void eligible_ptr(const Donor *list, int n, const char *group, int *count, int *youngest_id)
{
    int best_age = 1000;
    *count = 0; *youngest_id = -1;
    for (int i = 0; i < n; i++)
        if (strcmp(list[i].group, group) == 0 && is_eligible(&list[i])) {
            (*count)++;
            if (list[i].age < best_age) { best_age = list[i].age; *youngest_id = list[i].id; }
        }
}

/* Design 2: results returned together in a structure */
EligibleResult eligible_struct(const Donor *list, int n, const char *group)
{
    EligibleResult r = {0, -1};
    int best_age = 1000;
    for (int i = 0; i < n; i++)
        if (strcmp(list[i].group, group) == 0 && is_eligible(&list[i])) {
            r.count++;
            if (list[i].age < best_age) { best_age = list[i].age; r.youngest_id = list[i].id; }
        }
    return r;
}

int main(void)
{
    Donor list[] = {
        {7001, "Anil",   "O+", 34, 120}, {7002, "Bala",  "O+", 22, 200},
        {7003, "Chitra", "A+", 41,  30}, {7004, "Dev",   "O+", 19,  95},
        {7005, "Esha",   "O+", 70, 300}, {7006, "Farid", "O+", 28,  40}};
    int n = sizeof list / sizeof list[0];

    int count, youngest;
    eligible_ptr(list, n, "O+", &count, &youngest);
    printf("Design 1 (pointer params): count = %d, youngest eligible id = %d\n", count, youngest);

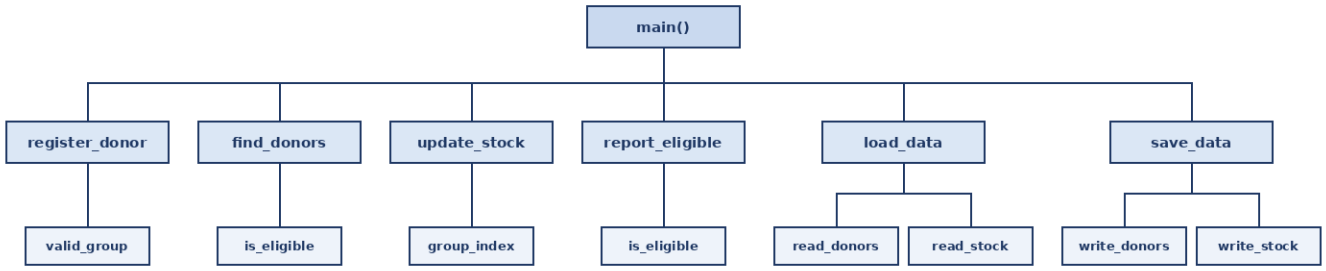
    EligibleResult r = eligible_struct(list, n, "O+");
    printf("Design 2 (struct return) : count = %d, youngest eligible id = %d\n", r.count, r.youngest_id);
    return 0;
}
```

A1.2 — Prototypes (value / pointer / const decisions)

```c
int  valid_group(const char *group);                /* string only read -> const pointer (an array decays to a pointer anyway)
        */
int  group_index(const char *group);                /* read-only text, returns index 0..7 or -1
        */
int  is_eligible(const Donor *d);                   /* one record only read: const pointer avoids copying the whole struct
        */
int  register_donor(DonorList *list, const Donor *d);/* list is modified -> plain pointer; d only read -> const pointer
        */
void eligible_ptr(const Donor *list, int n, const char *group,
                  int *count, int *youngest_id);    /* list/group read-only; count/id are OUTPUT pointers; n by value (small)
        */
EligibleResult eligible_struct(const Donor *list, int n, const char *group);  /* returns two values together                */
int  update_stock(Stock *stock, int g, int delta);  /* stock array is changed -> pointer; g, delta small ints -> by value
        */
int  load_data(const char *donor_file, const char *stock_file,
               DonorList *list, Stock stock[8]);    /* file names read-only; list and stock filled by the function
        */
int  save_data(const char *donor_file, const char *stock_file,
               const DonorList *list, const Stock stock[8]);   /* data only read -> const pointers
        */
/* find_donors is implemented by eligible_ptr / eligible_struct above; report_eligible, read_* and write_* follow the same
        rules: */
int  report_eligible(const DonorList *list, int days_ahead);              /* read-only list, small int by value
        */
int  read_donors(FILE *fp, DonorList *list);   int  write_donors(FILE *fp, const DonorList *list);
int  read_stock(FILE *fp, Stock stock[8]);     int  write_stock(FILE *fp, const Stock stock[8]);
```

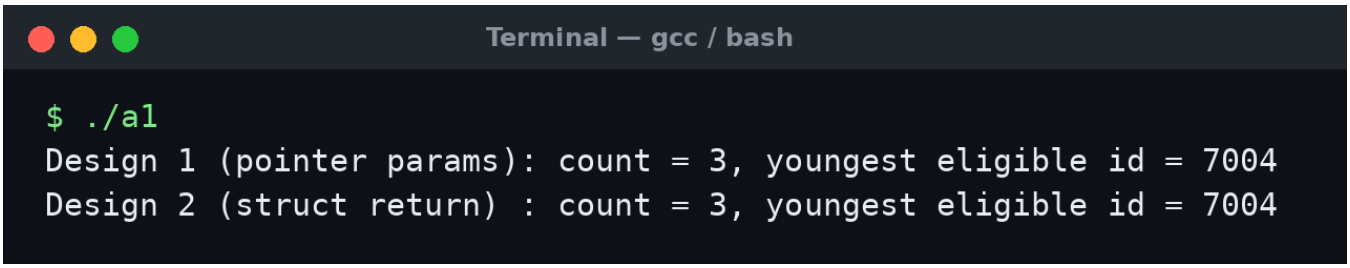
Output Screenshot

A1.1 — Structure chart



Level 1: driver | Level 2: one function per job | Level 3: helpers (is_eligible is shared by find_donors and report_eligible)

A line means 'calls' (parent above, child below). Data moves only through parameters; read-only data is passed as const pointers.



Analysis / Discussion

Output: both designs report 3 eligible O+ donors and the same youngest eligible donor (ID 7004, age 19), so the two return styles are equivalent. Donor 7005 (age 70) and 7006 (only 40 days since donation) are correctly excluded.

A1.2 justification: small scalars (n, g, delta) are passed by value because a copy is cheap and cannot damage the caller's data. Arrays and structures are passed by pointer, so only 4/8 bytes are copied instead of a whole record. A pointer gets const whenever the function does not write through it (is_eligible, eligible_*, save_data); it has no const only when the function must modify the data (register_donor's list, update_stock, load_data).

A1.3 comparison: output pointers keep the return value free for an error code and need no extra type, but the call site is harder to read and the callee can forget to set an output. A returned struct gives a self-documenting result, cannot be left half-filled, and is easy to pass on; the cost is one extra type and a small copy of the struct (8 bytes here, negligible).

Alternative — global variables: if the functions shared donors, stock and results through globals, the running time and memory would stay practically the same (O(n) scan, no extra arrays; at most a few pushed arguments are saved). What changes is software quality. Coupling rises because every function silently depends on the same hidden state. Testing becomes harder because each test must first reset the globals and cannot run in isolation or in parallel. Reuse falls because a function cannot be copied into another program without dragging the globals along. Parameters make the data flow visible, so the parameter version is preferred.

Result

Task A1 completed. The program was divided into single-purpose functions (structure chart and prototypes), every parameter has a justified passing mode, and both return-two-values designs were implemented and gave identical output. Learned: const pointers and structs make interfaces safe and readable, and globals trade away testability for no performance gain.

TASK A2 — Design with Recursion (5 Marks)

Task / Problem Statement

Donor IDs carry a check digit equal to the digital root of the other digits (sum the digits repeatedly until one digit remains). The blood bank also computes base^n for a reporting formula. A2.1: write recursive pseudocode for the digital root and for base^n using base^n = (base^(n/2))^2 and name the base and recursive case. A2.2: trace both for one input, draw the call stack at its deepest point and write the recurrence for the number of calls of linear and halving power. A2.3: explain a missing base case and when to replace recursion by a loop.

Aim

To design recursive solutions for the digital root and fast power, to trace their calls and recurrences, and to understand the risks of recursion.

Concept / Theory

Recursion solves a problem by calling the same function on a smaller input. Every correct recursive function needs a base case that stops the calls and a recursive case that moves toward it. Each call receives a stack frame holding its parameters and return address, so the stack depth equals the recursion depth. A recurrence counts the work: T(n) = T(n-1) + 1 gives n+1 calls (linear power) while T(n) = T(n/2) + 1 gives about log2(n) + 2 calls (halving power) because the problem size is halved each time.

Procedure / Algorithm

1. Define digital_root(n): if n < 10 return n (base case), otherwise return digital_root(sum of digits of n).
2. Define power_linear(base, n) = base * power_linear(base, n-1) with base case n = 0.
3. Define power_fast(base, n): compute half = power_fast(base, n/2) once, return half*half (times base if n is odd), base case n = 0.
4. Trace digital_root(9875) and power_fast(2, 10) and draw the call stack at the deepest point.
5. Write the recurrences T(n) for both power versions and solve them.
6. Run a demo program that prints the indented call trace and verify the results against the formula 1 + (n-1) % 9.

Program / Source Code

Pseudocode

```
ALGORITHM digital_root(n)
BEGIN
   IF n < 10 THEN RETURN n          // base case
   s <- sum_digits(n)
   RETURN digital_root(s)           // recursive case
END

ALGORITHM power_linear(base, n)
BEGIN
   IF n = 0 THEN RETURN 1                 // base case
   RETURN base * power_linear(base, n - 1) // recursive case
END

ALGORITHM power_fast(base, n)
BEGIN
   IF n = 0 THEN RETURN 1                         // base case
   half <- power_fast(base, n DIV 2)              // ONE recursive call
   IF n MOD 2 = 0 THEN RETURN half * half         // even exponent
   ELSE RETURN half * half * base                 // odd exponent
END
```

A2 — C program (call trace demo)

```c
#include <stdio.h>

static long long calls;
static int depth, maxdepth;

static int sum_digits(int n) { int s = 0; while (n) { s += n % 10; n /= 10; } return s; }

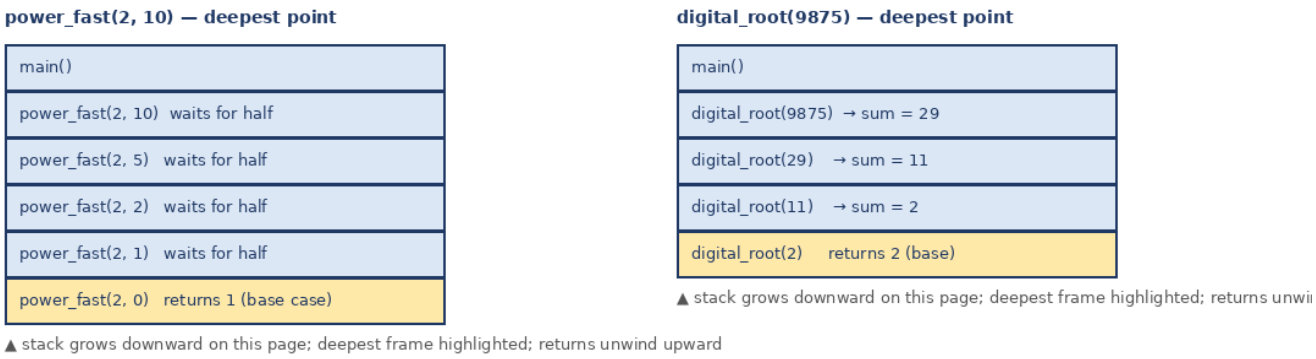
int digital_root(int n)
{
    printf("%*sdigital_root(%d)\n", depth * 2, "", n);
    if (n < 10) return n;                /* base case */
    depth++;
    int r = digital_root(sum_digits(n));  /* recursive case */
    depth--;
    return r;
}

long long power_fast(long long base, int exp)
{
    calls++; depth++; if (depth > maxdepth) maxdepth = depth;
    printf("%*spower_fast(%lld, %d)\n", (depth - 1) * 2, "", base, exp);
    long long result;
    if (exp == 0) result = 1;            /* base case */
    else {
        long long half = power_fast(base, exp / 2);
        result = half * half;
        if (exp % 2) result *= base;
    }
    depth--;
    return result;
}

int main(void)
{
    puts("--- digital_root(9875) ---");
    int dr = digital_root(9875);
    printf("result = %d, formula 1+(n-1)%%9 = %d\n\n", dr, 1 + (9875 - 1) % 9);
    puts("--- power_fast(2, 10) call stack ---");
    depth = 0; calls = 0;
    long long p = power_fast(2, 10);
    printf("result = %lld, calls = %lld, deepest stack = %d frames\n", p, calls, maxdepth);
    return 0;
}
```

Output Screenshot

A2.2 — Call stack at the deepest point



Terminal — gcc / bash

```
$ ./a2
--- digital_root(9875) ---
digital_root(9875)
  digital_root(29)
    digital_root(11)
      digital_root(2)
result = 2, formula 1+(n-1)%9 = 2

--- power_fast(2, 10) call stack ---
power_fast(2, 10)
  power_fast(2, 5)
    power_fast(2, 2)
      power_fast(2, 1)
        power_fast(2, 0)
result = 1024, calls = 5, deepest stack = 5 frames
```

Analysis / Discussion

Trace: digital_root(9875) -> 9+8+7+5 = 29 -> 2+9 = 11 -> 1+1 = 2 -> returns 2 after 4 calls, so the deepest stack holds main plus 4 frames. power_fast(2, 10) calls n = 10, 5, 2, 1, 0 (5 calls); the deepest frame is power_fast(2, 0) which returns 1, and the values 1, 2, 4, 32 and finally 1024 are rebuilt while the stack unwinds.

Recurrences: linear power T(n) = T(n-1) + 1, T(0) = 1, so T(n) = n + 1 calls (O(n)). Halving power T(n) = T(n/2) + 1, T(0) = 1, so T(n) = floor(log2 n) + 2 calls (O(log n)); for n = 10 this gives 5, matching the trace.

A2.3 — If the base case is missing, the function never stops calling itself, every call adds a stack frame, and the program ends with a stack overflow (segmentation fault). Recursion should be replaced by a loop when the depth can become very large (e.g. linear power for n in the hundreds of thousands), when each call does almost no work, or when the stack space matters; a loop uses constant stack.

Alternative — digital root by loop or formula: a loop (while n >= 10: n = sum_digits(n)) does the same digit sums but uses O(1) stack instead of one frame per round. Time is proportional to the number of digits d (the first digit sum costs O(d), later ones are much smaller). The closed formula 1 + (n - 1) % 9 (for n > 0, and 0 for n = 0) needs a single division, so time O(1) and space O(1). Recursion adds O(number of rounds) stack space, which is tiny here (at most 3-4 frames for a 32-bit ID) but is still the most expensive of the three.

Result

Task A2 completed. Both recursive designs have a clear base case and recursive case, the traces and recurrences agree with the program output, and the digital root agrees with the O(1) formula. Learned: halving the exponent turns an O(n) recursion into O(log n), and a loop or formula is safer when the depth is unbounded.

TASK A3 — Design with Pointers (5 Marks)

Task / Problem Statement

Given int stock[8] (one entry per blood group) and int *p = stock;, A3.1 explains p + 3, *(p + 3), p[3] and *p++ with a memory diagram. A3.2 designs a function that returns a pointer to the blood group with the lowest stock so the caller can update it directly, and explains why returning a pointer to a local variable is wrong. A3.3 designs the command-line interface (argc / argv) of the program and how invalid arguments are handled.

Aim

To use pointer arithmetic correctly, to design a safe pointer-returning function, and to design a command-line interface with proper validation.

Concept / Theory

The name of an array decays to the address of its first element, so p points to stock[0]. Adding an integer to a pointer moves it by that many elements, not bytes, because the compiler multiplies by sizeof(int). *p reads the object pointed to, and p[i] is defined as *(p + i). A pointer returned from a function must point to storage that outlives the call (caller's array, static or heap); a local variable is destroyed when the function returns, leaving a dangling pointer. main(int argc, char *argv[]) receives the program name and the command-line words as strings.

Procedure / Algorithm

1. Draw int stock[8] with the address of every element and show p, p + 3 and p after *p++.
2. Write lowest_stock(arr, n) that keeps a pointer to the smallest element and returns it.
3. Explain why a pointer to a local variable cannot be returned and how to avoid it.
4. Design the commands: bloodbank find <group>, bloodbank issue <group> <units>.
5. List the validation rules: argument count, valid group name, positive integer units, file errors.
6. Run a demo program to verify the pointer expressions and the pointer / index versions.

Program / Source Code

Pseudocode

```
ALGORITHM lowest_stock(arr, n) RETURNS pointer to integer
BEGIN
   low <- address of arr[0]
   FOR i <- 1 TO n-1 DO
      IF arr[i] < value at low THEN low <- address of arr[i]
   RETURN low                                  // points into the CALLER's array
END

ALGORITHM main(argc, argv)                      // command-line interface
BEGIN
   IF argc < 2 THEN print usage ; EXIT 2
   IF argv[1] = 'find'  THEN  require argc = 3 ; g <- group_index(argv[2]) ; IF g < 0 THEN error ; print stock[g]
   IF argv[1] = 'issue' THEN  require argc = 4 ; check group ; u <- strtol(argv[3]) ; IF u <= 0 THEN error ;
                              IF stock[g] < u THEN 'not enough stock' ELSE stock[g] <- stock[g] - u
   ELSE print usage ; EXIT 2                     // unknown command
END      Exit codes: 0 = success, 1 = file or stock error, 2 = bad arguments
```

A3 — C program (pointer expressions and lowest stock)

```c
#include <stdio.h>

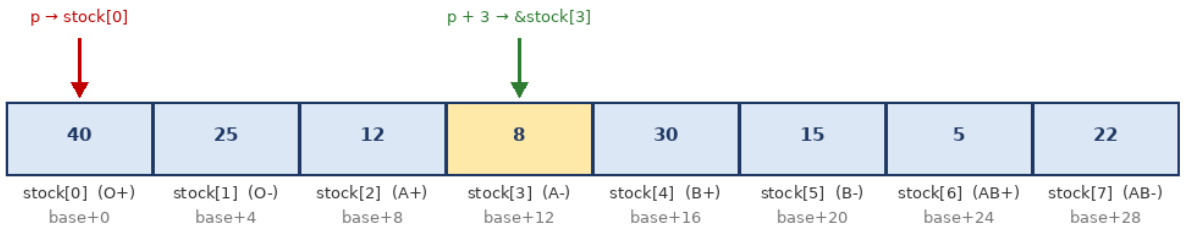
int *lowest_stock(int *arr, int n)          /* returns pointer INTO caller's array */
{
    int *low = arr;
    for (int *q = arr + 1; q < arr + n; q++) if (*q < *low) low = q;
    return low;
}
int lowest_index(const int *arr, int n)     /* alternative: returns index */
{
    int k = 0;
    for (int i = 1; i < n; i++) if (arr[i] < arr[k]) k = i;
    return k;
}

int main(void)
{
    int stock[8] = {40, 25, 12, 8, 30, 15, 5, 22};   /* O+ O- A+ A- B+ B- AB+ AB- */
    int *p = stock;
    printf("stock = %p, p = %p\n", (void *)stock, (void *)p);
    printf("p + 3     = %p  (address of stock[3], 3 ints = %ld bytes ahead)\n", (void *)(p + 3), (long)((char *)(p + 3) - (char
        *)p));
    printf("*(p + 3)  = %d  (value of stock[3])\n", *(p + 3));
    printf("p[3]      = %d  (same as *(p + 3))\n", p[3]);
    int v = *p++;
    printf("*p++      = %d  (old element used, then p moves to stock[1]); now *p = %d\n\n", v, *p);

    int *low = lowest_stock(stock, 8);
    printf("lowest via pointer: index %ld holds %d\n", (long)(low - stock), *low);
    *low += 10;                                  /* caller updates directly */
    printf("after *low += 10  : stock[6] = %d\n", stock[6]);
    int k = lowest_index(stock, 8);
    stock[k] += 10;                              /* index version */
    printf("lowest via index  : now index %d holds %d (stock[%d] = %d)\n", k, stock[k], k, stock[k]);
    return 0;
}
```

Output Screenshot

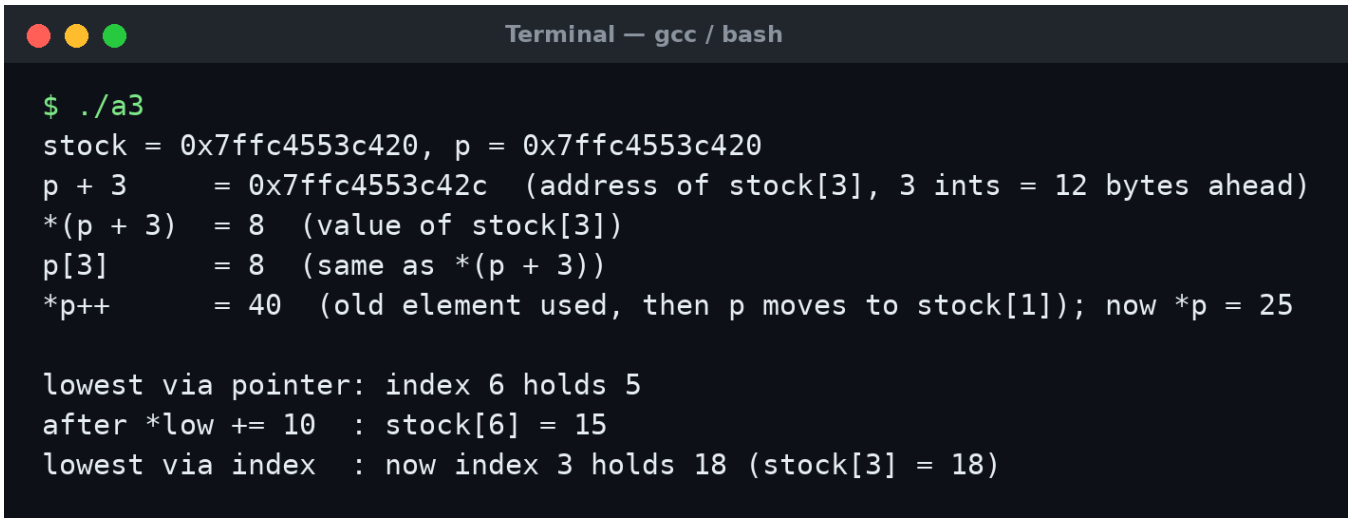
A3.1 — Memory diagram



*(p + 3) = p[3] = stock[3] = 8 *p++ → the value 40 is used, then p advances one int (4 bytes) to stock[1]

p + 3 is an ADDRESS (base + 3×sizeof(int) = base + 12); *(p + 3) and p[3] are the VALUE stored there.

Pointer arithmetic is scaled by the element size, so p + 1 moves 4 bytes, not 1 byte.



Analysis / Discussion

A3.1: p + 3 is the address of stock[3] (base + 12 bytes); *(p + 3) and p[3] are both the value 8 stored there (p[3] is just another way of writing *(p + 3)); *p++ reads the value 40 and afterwards moves p to stock[1] (postfix ++ has higher precedence than *, but the old pointer value is what gets dereferenced). The output confirms: 12 bytes ahead, value 8, value 8, value 40 then *p = 25.

A3.2: lowest_stock returned a pointer to stock[6] (value 5); the caller wrote *low += 10 and the array changed directly, with no search needed again. Returning a pointer to a local variable is wrong because the local lives in the function's stack frame; after return the frame is reused, so the pointer is dangling and reading or writing it is undefined behaviour (it may show garbage or crash).

A3.3: invalid input is rejected before any file is touched: no arguments or an unknown command prints the usage line (exit 2); wrong argument count (too few or too many) prints usage; an invalid group name such as Z9 or a non-numeric / non-positive unit count prints a clear message (exit 2); file problems are reported with perror() (exit 1). This design is implemented and tested in Task B2.

Alternative — returning an index: the caller writes int k = lowest_index(stock, 8); stock[k] += 10; instead of int *low = lowest_stock(stock, 8); *low += 10;. Both scan the array once, so time is O(n) and extra space is O(1). The index version is safer if the array is later reallocated: realloc may move the block, so a stored pointer then points to freed memory (dangling) whereas an index is still valid because it is applied to the new base address. The pointer version is faster to use after the search and avoids repeating stock[k].

Result

Task A3 completed. The pointer expressions were explained and verified, a safe pointer-returning function was designed, and the command-line interface with its error handling was specified. Learned: pointer arithmetic is scaled by element size, and a returned pointer must never refer to a local variable.

TASK A4 — Design Dynamic Memory Use (4 Marks)

Task / Problem Statement

The number of donors is unknown: 50 in a small hospital or 50,000 in the district. A4.1: design a donor list that starts small and doubles its capacity with realloc() when full, with pseudocode for adding a donor and the failure case. A4.2: explain a memory leak, a dangling pointer and a double free with an example from the blood bank and how the design prevents each. Alternative: grow by a fixed 10 records and compare the copying cost and the unused space.

Aim

To design a growable array with safe error handling and to understand and prevent the three common dynamic-memory errors.

Concept / Theory

malloc, realloc and free manage heap memory whose size is chosen at run time. realloc may move the block, copying the old elements, and returns NULL if it fails (the old block stays valid). Doubling the capacity makes the copying cost amortised O(1) per insertion, because the total copied is 1 + 2 + 4 + ... < 2n. A leak is memory that is no longer reachable but never freed; a dangling pointer refers to memory that was freed or moved; a double free releases the same block twice and corrupts the heap.

Procedure / Algorithm

1. Represent the list as a struct {data, size, capacity} starting with data = NULL, size = 0, capacity = 0.
2. When size == capacity compute new capacity (2 if 0, else double).
3. Call realloc with a temporary pointer; if it returns NULL keep the old list and report failure.
4. Store the new pointer and capacity, then append the donor and increase size.
5. Explain leak, dangling pointer and double free and how each is prevented.
6. Count the elements copied by doubling and by +10 growth for n = 1,000, 10,000 and 100,000 with a program.

Program / Source Code

Pseudocode

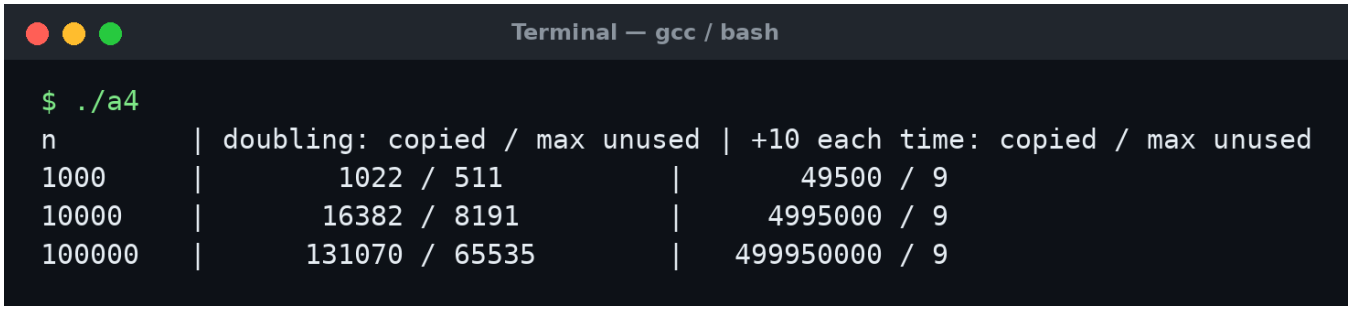
```
ALGORITHM add_donor(list, id, group) RETURNS 1 on success, 0 on failure
BEGIN
   IF list.size = list.capacity THEN
      IF list.capacity = 0 THEN new_cap <- 2 ELSE new_cap <- 2 * list.capacity
      p <- realloc(list.data, new_cap * size of Donor)        // temporary pointer
      IF p = NULL THEN RETURN 0                               // old data still valid, nothing lost
      list.data <- p ; list.capacity <- new_cap
   list.data[list.size] <- (id, group) ; list.size <- list.size + 1
   RETURN 1
END
```

A4 — C program (growth cost: doubling vs +10)

```c
#include <stdio.h>

int main(void)
{
    int sizes[] = {1000, 10000, 100000};
    printf("%-8s | %-26s | %-26s\n", "n", "doubling: copied / max unused", "+10 each time: copied / max unused");
    for (int s = 0; s < 3; s++) {
        int n = sizes[s];
        long long copied_d = 0, copied_f = 0; int cap_d = 0, cap_f = 0; int maxu_d = 0, maxu_f = 0;
        for (int size = 0; size < n; size++) {
            if (size == cap_d) { copied_d += size; cap_d = cap_d ? cap_d * 2 : 2; }
            if (size == cap_f) { copied_f += size; cap_f += 10; }
            if (cap_d - (size + 1) > maxu_d) maxu_d = cap_d - (size + 1);
            if (cap_f - (size + 1) > maxu_f) maxu_f = cap_f - (size + 1);
        }
        printf("%-8d | %11lld / %-12d | %11lld / %-12d\n", n, copied_d, maxu_d, copied_f, maxu_f);
    }
    return 0;
}
```

Output Screenshot



Analysis / Discussion

A4.2 Memory errors. Leak: loading donors into a list and exiting a function without free(list.data), or overwriting list->data = realloc(list->data, ...) when realloc fails, loses the only pointer to the old block. Prevented by freeing exactly once at the end and by using a temporary pointer. Dangling pointer: keeping Donor *d = &list.data[5] and then adding donors; if realloc moves the block, d points to freed memory. Prevented by keeping an index (or re-fetching the pointer) after any add. Double free: calling free(list.data) in both a cleanup function and main. Prevented by one owner of the memory and by setting list.data = NULL after free (free(NULL) is harmless).

Alternative — growing by 10: every growth copies all current elements, so the total copied is 0 + 10 + 20 + ... = n(n-10)/20 which is O(n^2) (measured 499,950,000 element copies for n = 100,000) whereas doubling copies fewer than 2n elements, O(n) overall (131,070 for n = 100,000, i.e. about 3,800 times less). The price of doubling is wasted space: up to about half of the capacity can be unused right after a doubling (65,535 free slots measured at n = 100,000) while the fixed +10 strategy wastes at most 9 slots. Doubling therefore trades some memory for far less time, which is the right choice for a list that may reach 50,000 donors.

Result

Task A4 completed. A growable donor list with failure handling was designed, the three memory errors were explained with prevention, and the measured copy counts proved that doubling is O(n) while fixed growth is O(n^2). Learned: always keep the old pointer until realloc succeeds.

TASK A5 — Design the File Storage (4 Marks)

Task / Problem Statement

A5.1: design the donor file as a text file (one donor per line, written with fprintf) and the stock file as a binary file of fixed-size records (written with fwrite), and justify both choices. A5.2: write pseudocode to change the units of one blood group in the binary stock file with fseek, fread and fwrite without rewriting the file, and explain why this is hard for a text file. Alternative: keep the stock in a text file and rewrite the whole file on every update.

Aim

To choose suitable file formats for different kinds of data and to design an in-place update of one record.

Concept / Theory

A text file stores characters, so numbers are converted with fprintf/fscanf; it is human-readable and portable but lines have variable length. A binary file stores the raw bytes of a structure with fwrite/fread; with fixed-size records, record i starts at offset i * sizeof(record), so fseek can jump straight to it (direct access). Changing a number inside a text line can change the line length, which shifts every later byte.

Procedure / Algorithm

1. Donor file: one line per donor 'id,name,group,age,phone,date' written with fprintf.
2. Stock file: struct Stock {group_code, units} (8 bytes) written with fwrite, one record per blood group.
3. In-place update: compute offset = index * sizeof(Stock) and fseek to it.
4. fread the record, change units, fseek back one record and fwrite it.
5. Check the return value of every call and report failure.
6. Run a demo that compares file sizes and shows the in-place update.

Program / Source Code

Pseudocode

```
ALGORITHM update_stock_in_place(file, g, new_units) RETURNS success or failure
BEGIN
   OPEN file in binary read/update mode ; IF open fails THEN RETURN failure
   SEEK to position g * size of Stock from the start of the file
   READ one Stock record into s ; IF read fails THEN CLOSE ; RETURN failure
   s.units <- new_units
   SEEK back by one record (-size of Stock)
   WRITE s ; CLOSE file ; RETURN success
END
```

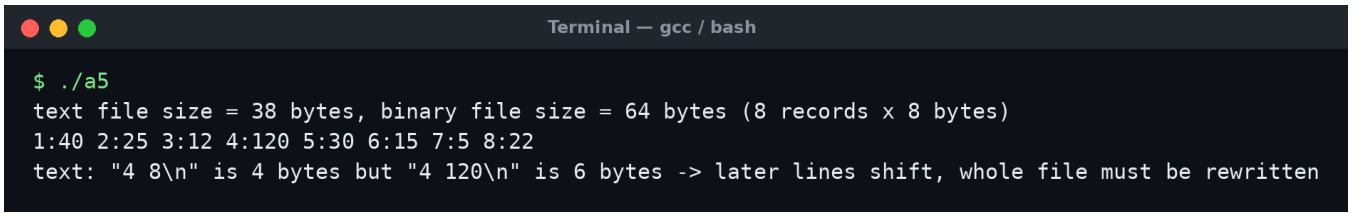
A5 — C program (text vs binary stock file)

```
#include <stdio.h>
#include <string.h>
typedef struct { int group_code; int units; } Stock;

int main(void)
{
    Stock s[8] = {{1,40},{2,25},{3,12},{4,8},{5,30},{6,15},{7,5},{8,22}};
    FILE *t = fopen("stock.txt", "w"), *b = fopen("stock.bin", "wb");
    for (int i = 0; i < 8; i++) fprintf(t, "%d %d\n", s[i].group_code, s[i].units);
    fwrite(s, sizeof(Stock), 8, b);
    fclose(t); fclose(b);
    b = fopen("stock.bin", "rb"); fseek(b, 0, SEEK_END); long bsz = ftell(b); fclose(b);
    t = fopen("stock.txt", "rb"); fseek(t, 0, SEEK_END); long tsz = ftell(t); fclose(t);
    printf("text file size = %ld bytes, binary file size = %ld bytes (8 records x %zu bytes)\n", tsz, bsz, sizeof(Stock));

    /* in-place update of record 3 (units 8 -> 120) */
    b = fopen("stock.bin", "r+b");
    Stock r; fseek(b, 3L * sizeof(Stock), SEEK_SET); fread(&r, sizeof r, 1, b);
    r.units = 120;
    fseek(b, -(long)sizeof(Stock), SEEK_CUR); fwrite(&r, sizeof r, 1, b);
    fseek(b, 0, SEEK_SET);
    for (int i = 0; i < 8; i++) { fread(&r, sizeof r, 1, b); printf("%d:%d ", r.group_code, r.units); }
    printf("\n");
    fclose(b);
    printf("text: \"4 8\\n\" is 4 bytes but \"4 120\\n\" is 6 bytes -> later lines shift, whole file must be rewritten\n");
    return 0;
}
```

Output Screenshot



Analysis / Discussion

A5.1 Justification. The donor file is text because donor data is variable length (names, phone numbers), people and other programs can read, edit, diff and back it up easily, it is portable between computers, and new donors are simply appended. The stock file is binary with fixed-size records because there are only 8 groups, the data is updated after every donation or issue, and fixed records give direct access without parsing.

A5.2: in the binary file the update touches only 8 bytes at a known offset (the demo changed record 3 from 8 to 120 units and the other records stayed unchanged). In a text file the line '4 8' is 4 bytes but '4 120' is 6 bytes, so all following lines must move; the usual

remedy is to read the whole file, change the line and write a new file.

Alternative — rewrite the text file for every update: the time per update is O(n) because all n records are read and written, against O(1) for fseek + fread + fwrite. Measured on 100,000 records in Task B3 this is about 36 ms versus 0.011 ms. File size: for tiny numbers the text file is smaller (38 bytes against 64 bytes here) because it stores only the digits; the binary record is always 8 bytes per record. For large numbers or many fields the binary size is predictable, while text length varies with the digits.

Result

Task A5 completed. Text format was chosen for donors and fixed-size binary records for stock, the in-place update pseudocode was written and verified in a demo, and the cost of rewriting a text file was analysed. Learned: fixed-size records make random access and in-place update possible.

TASK B1 — Implement and Instrument the Recursive Functions (7 Marks)

Task / Problem Statement

B1.1: implement the digital root and both power functions and test them with ordinary values, 0, 1 and one value of my choice. B1.2: compare the code with the Part A pseudocode. B1.3: count the recursive calls of linear and halving power for n = 10, 100, 1,000 and 10,000 and compare with the recurrences. B1.4: increase n for the linear version until the program crashes, record the value and explain it, then repeat with an iterative version. Complete TODO 1 (power_fast) so that the expected output 'linear ... in 31 calls / fast ... in 6 calls' is produced, and replace half * half by two recursive calls to record the new call count.

Aim

To implement and test the recursive functions, to measure their number of calls against the recurrences, and to find the recursion depth limit of the machine.

Concept / Theory

A global counter incremented at the start of each function counts the calls without changing the result (instrumentation). Linear recursion has depth n, halving recursion has depth about log2 n. Each active call occupies a stack frame of a fixed size, and the stack of a process is limited (8 MB on this system); exceeding it causes a stack overflow and the operating system terminates the program with a segmentation fault. A loop does not add frames.

Procedure / Algorithm

1. Complete TODO 1: if exp == 0 return 1; half = power_fast(base, exp/2) once; return half*half, times base when exp is odd.
2. Add digital_root and a variant power_twice that makes two recursive calls instead of one.
3. Test: ordinary values, 0, 1 and a chosen value (digital_root of 9875 and 123456789, power(3,7)).
4. Print call counts for n = 10, 100, 1000, 10000 using base 1 (the count does not depend on the base).
5. In b1_limit, run power_linear in a child process for increasing n until the child dies of a signal, then narrow the limit with a binary search.
6. Run an iterative power for a very large n to show that it does not crash.

Program / Source Code

B1 — Completed program (TODO 1 done) — b1.c

```
#include <stdio.h>

long long calls = 0; /* number of recursive calls made */

/* Linear recursion: base^exp = base * base^(exp-1) */
long long power_linear(long long base, int exp)
{
    calls++;
    if (exp == 0) return 1;
    return base * power_linear(base, exp - 1);
}

/* Divide-and-conquer recursion: compute base^(exp/2) once and square it. */
long long power_fast(long long base, int exp)
{
    calls++;
    if (exp == 0) return 1;                      /* TODO 1: base case */
    long long half = power_fast(base, exp / 2);  /* computed ONCE */
    if (exp % 2 == 0) return half * half;
    return half * half * base;                   /* odd exponent */
}

/* Experiment: two recursive calls instead of one */
long long power_twice(long long base, int exp)
{
    calls++;
    if (exp == 0) return 1;
    long long r = power_twice(base, exp / 2) * power_twice(base, exp / 2);
    return (exp % 2) ? r * base : r;
}

int digital_root(int n) { if (n < 10) return n; int s = 0; while (n) { s += n % 10; n /= 10; } return digital_root(s); }

int main(void)
{
    calls = 0;
    long long a = power_linear(2, 30);
    printf("linear: %lld in %lld calls\n", a, calls);
    calls = 0;
    long long b = power_fast(2, 30);
    printf("fast : %lld in %lld calls\n", b, calls);
    calls = 0;
    long long c = power_twice(2, 30);
    printf("twice (half*half replaced by 2 calls): %lld in %lld calls\n\n", c, calls);

    puts("B1.1 validation");
    printf("digital_root(0)=%d  (1)=%d  (9875)=%d  (123456789)=%d\n", digital_root(0), digital_root(1), digital_root(9875),
        digital_root(123456789));
    printf("power(5,0): linear=%lld fast=%lld | power(5,1): linear=%lld fast=%lld | power(3,7): linear=%lld fast=%lld\n\n",
           power_linear(5,0), power_fast(5,0), power_linear(5,1), power_fast(5,1), power_linear(3,7), power_fast(3,7));

    puts("B1.3 call counts (base 1 so no overflow; counts do not depend on base)");
    printf("%-8s %-14s %-14s\n", "n", "linear calls", "halving calls");
    int ns[] = {10, 100, 1000, 10000};
    for (int i = 0; i < 4; i++) {
        calls = 0; power_linear(1, ns[i]); long long l = calls;
        calls = 0; power_fast(1, ns[i]);   long long f = calls;
        printf("%-8d %-14lld %-14lld\n", ns[i], l, f);
    }
    return 0;
}
```

B1.4 — Finding the recursion limit — b1_limit.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>

long long power_linear(long long base, int exp) { if (exp == 0) return 1; return base * power_linear(base, exp - 1); }
long long power_iter(long long base, int exp) { long long r = 1; for (int i = 0; i < exp; i++) r *= base; return r; }

static int crashes(int n)            /* run in a child so the crash does not kill us */
{
    pid_t pid = fork();
    if (pid == 0) { power_linear(1, n); _exit(0); }
    int st; waitpid(pid, &st, 0);
    return WIFSIGNALED(st);
}

int main(void)
{
    struct rlimit rl; getrlimit(RLIMIT_STACK, &rl);
    printf("stack limit = %lu KB\n", (unsigned long)rl.rlim_cur / 1024);
    int n = 1000;
    while (!crashes(n)) { printf("linear n = %-9d OK\n", n); n *= 2; }
    printf("linear n = %-9d CRASH (SIGSEGV: stack overflow)\n", n);
    int lo = n / 2, hi = n;
    while (hi - lo > 1000) { int mid = (lo + hi) / 2; if (crashes(mid)) hi = mid; else lo = mid; }
    printf("limit is between %d and %d\n", lo, hi);
    printf("iterative n = 2000000000 -> power_iter(1, n) = %lld (no crash)\n", power_iter(1, 2000000000));
    return 0;
}
```

Output Screenshot

Terminal — gcc / bash

```
$ ./b1
linear: 1073741824 in 31 calls
fast : 1073741824 in 6 calls
twice (half*half replaced by 2 calls): 1073741824 in 63 calls

B1.1 validation
digital_root(0)=0  (1)=1  (9875)=2  (123456789)=9
power(5,0): linear=1 fast=1 | power(5,1): linear=5 fast=5 | power(3,7): linear=2187 fast=2187

B1.3 call counts (base 1 so no overflow; counts do not depend on base)
n        linear calls   halving calls
10       11             5
100      101            8
1000     1001           11
10000    10001          15
$ ./b1_limit
stack limit = 8192 KB
linear n = 1000      OK
linear n = 2000      OK
linear n = 4000      OK
linear n = 8000      OK
linear n = 16000     OK
linear n = 32000     OK
linear n = 64000     OK
linear n = 128000    OK
linear n = 256000    OK
linear n = 512000    CRASH (SIGSEGV: stack overflow)
limit is between 261000 and 262000
iterative n = 2000000000 -> power_iter(1, n) = 1 (no crash)
```

Analysis / Discussion

Expected output reproduced exactly: linear 1073741824 in 31 calls (n + 1 = 31) and fast 1073741824 in 6 calls (floor(log2 30) + 2 = 6). Replacing half * half with two calls power_fast(base, exp/2) * power_fast(base, exp/2) gave 63 calls: the recurrence becomes T(n) = 2T(n/2) + 1, whose call tree has 6 levels of doubling width (2^6 - 1 = 63), so the benefit of halving is lost and the cost grows to O(n) again, as bad as the linear version.

B1.1 tests: ordinary values (digital_root(9875) = 2, power(3,7) = 2187 for both versions) verify the general case; 0 (digital_root(0) = 0, power(5,0) = 1) verifies the base case; 1 (digital_root(1) = 1, power(5,1) = 5) verifies the smallest recursive step; 123456789 -> 9 is my own value and verifies a multi-round digit sum.

B1.2: the code follows the Part A pseudocode almost line by line. The only change is that digit summing is written as an inner while loop (sum_digits) instead of recursion, and the call counter calls++ is added to instrument the functions.

B1.3 call counts: n = 10 -> 11 / 5, n = 100 -> 101 / 8, n = 1,000 -> 1,001 / 11, n = 10,000 -> 10,001 / 15 (linear / halving). They equal n + 1 and floor(log2 n) + 2 from the A2.2 recurrences in every row.

B1.4 limit: with the default 8,192 KB stack the linear version ran correctly up to n = 256,000 and crashed with SIGSEGV at 512,000; the binary search located the limit between 261,000 and 262,000 calls, i.e. about 8 MB / 261,500 = 32 bytes per frame. The crash is a stack overflow: each call keeps base, exp and a return address on the stack and the stack segment is exhausted. The iterative version computed n = 2,000,000,000 without any problem (constant stack).

Alternative — iterative square-and-multiply: result = 1; while (exp > 0) { if (exp & 1) result *= base; base *= base; exp >>= 1; }. Time is O(log n) multiplications, the same as the halving recursion, but the stack space is O(1) instead of O(log n) frames. Compared with the linear recursion (O(n) time and O(n) stack) both are far better, and only the iterative form has no depth limit.

Result

Task B1 completed. TODO 1 was completed and the expected output was obtained, the call counts matched the recurrences exactly, the two-call experiment proved that computing the half only once is essential, and the stack-overflow limit of linear recursion was measured (about 261,000 calls).

TASK B2 — Engineer Adversarial Inputs (5 Marks)

Task / Problem Statement

Give the program a donor name longer than the array that holds it, first with scanf("%s") and then with fgets, and explain what happens. Run it with a missing or read-only data file and show that it reports the error with perror() instead of crashing. Pass wrong command-line arguments (none, too many, an invalid blood group). Run it under valgrind or with -fsanitize=address and report and fix memory errors. Alternative: read lines with getline().

Aim

To test the blood bank program with deliberately bad input, observe how it fails, and correct it so that it fails safely.

Concept / Theory

scanf("%s") has no limit, so input longer than the array writes beyond it (buffer overflow), which corrupts the stack; modern compilers detect it with a stack protector and AddressSanitizer reports it precisely. fgets(buf, size, stdin) never writes more than size - 1 characters plus the terminator. fopen returns NULL when a file cannot be opened and errno describes the reason; perror() prints it in plain words. getline() allocates the buffer it needs, so the caller must free it. valgrind and AddressSanitizer detect invalid accesses and leaks at run time.

Procedure / Algorithm

1. Write bloodbank.c with the commands find, issue, name-scanf, name-fgets and name-getline and a binary stock.dat.
2. Feed a 40-character name to name-scanf (name array of 8 chars) in a normal build and in an AddressSanitizer build.
3. Feed the same name to name-fgets and name-getline.
4. Run with a missing stock file and with a read-only file (as another user) and check the perror() message.
5. Run with no arguments, too many arguments and an invalid group (Z9) and check the usage / error messages and exit codes.
6. Run the fixed versions under AddressSanitizer / valgrind and confirm zero errors and zero leaks.

Program / Source Code

B2 — bloodbank.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { int group_code; int units; } Stock;
static const char *GROUPS[8] = {"O+", "O-", "A+", "A-", "B+", "B-", "AB+", "AB-"};

static int group_index(const char *g) { for (int i = 0; i < 8; i++) if (strcmp(g, GROUPS[i]) == 0) return i; return -1; }

static int usage(void)
{
    fprintf(stderr, "usage: bloodbank find <group> | issue <group> <units> | name-scanf | name-fgets | name-getline\n");
    return 2;
}

static int load_stock(const char *path, Stock *s)
{
    FILE *fp = fopen(path, "rb");
    if (fp == NULL) { perror(path); return 0; }              /* report, do not crash */
    size_t n = fread(s, sizeof(Stock), 8, fp);
    fclose(fp);
    if (n != 8) { fprintf(stderr, "%s: file is damaged (%zu of 8 records)\n", path, n); return 0; }
    return 1;
}

int main(int argc, char *argv[])
{
    if (argc < 2) return usage();                                   /* none */

    if (strcmp(argv[1], "name-scanf") == 0) {
        char name[8];
        printf("Enter donor name: ");
        scanf("%s", name);                                          /* UNSAFE: no width limit */
        printf("stored name = %s\n", name);
        return 0;
    }
    if (strcmp(argv[1], "name-fgets") == 0) {
        char name[8];
        printf("Enter donor name: ");
        if (fgets(name, sizeof name, stdin) == NULL) return 1;      /* SAFE: at most 7 chars + '\0' */
        name[strcspn(name, "\n")] = '\0';
        printf("stored name = %s\n", name);
        return 0;
    }
    if (strcmp(argv[1], "name-getline") == 0) {
        char *line = NULL; size_t cap = 0;
        printf("Enter donor name: ");
        ssize_t len = getline(&line, &cap, stdin);                  /* allocates what is needed */
        if (len < 0) { free(line); return 1; }
        line[strcspn(line, "\n")] = '\0';
        printf("stored name = %s (length %zu, buffer %zu bytes)\n", line, strlen(line), cap);
        free(line);                                                 /* caller must free */
        return 0;
    }

    const char *file = getenv("STOCK_FILE") ? getenv("STOCK_FILE") : "stock.dat";
    Stock s[8];
    if (strcmp(argv[1], "find") == 0) {
        if (argc != 3) return usage();                              /* too many / too few */
        int g = group_index(argv[2]);
        if (g < 0) { fprintf(stderr, "invalid blood group '%s'\n", argv[2]); return 2; }
        if (!load_stock(file, s)) return 1;
        printf("%s: %d units\n", GROUPS[g], s[g].units);
        return 0;
    }
    if (strcmp(argv[1], "issue") == 0) {
        if (argc != 4) return usage();
        int g = group_index(argv[2]);
        if (g < 0) { fprintf(stderr, "invalid blood group '%s'\n", argv[2]); return 2; }
        char *end; long u = strtol(argv[3], &end, 10);
        if (*end != '\0' || u <= 0) { fprintf(stderr, "invalid unit count '%s'\n", argv[3]); return 2; }
        FILE *fp = fopen(file, "r+b");                              /* needs write permission */
        if (fp == NULL) { perror(file); return 1; }
        fseek(fp, (long)g * sizeof(Stock), SEEK_SET);
        Stock r; if (fread(&r, sizeof r, 1, fp) != 1) { fclose(fp); return 1; }
        if (r.units < u) { fprintf(stderr, "not enough stock: %d < %ld\n", r.units, u); fclose(fp); return 1; }
        r.units -= (int)u;
        fseek(fp, -(long)sizeof r, SEEK_CUR); fwrite(&r, sizeof r, 1, fp);
        fclose(fp);
        printf("issued %ld units of %s, %d left\n", u, GROUPS[g], r.units);
        return 0;
    }
    return usage();
}
```

Output Screenshot

Terminal — gcc / bash

```
$ echo ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789ABCD | ./bloodbank name-scanf
*** stack smashing detected ***: terminated
Aborted (core dumped)   [exit status 134]
$ echo ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789ABCD | ./bloodbank_asan name-scanf
=================================================================
==621==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7fd1d0200108 at pc 0x7fd1d2491ad0 b...
WRITE of size 41 at 0x7fd1d0200108 thread T0
    #0 0x7fd1d2491acf in scanf_common ../../../../src/libsanitizer/sanitizer_common/sanitizer_common_i...
    #1 0x7fd1d24cd542 in __isoc23_vscanf ../../../../src/libsanitizer/sanitizer_common/sanitizer_commo...
SUMMARY: AddressSanitizer: stack-buffer-overflow in main (bloodbank.c:34)
$ echo ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789ABCD | ./bloodbank name-fgets
Enter donor name: stored name = ABCDEFG

$ echo ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789ABCD | ./bloodbank name-getline
Enter donor name: stored name = ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789ABCD (length 40, buffer 120 bytes)
$ STOCK_FILE=missing.dat ./bloodbank find O+
missing.dat: No such file or directory
$ chmod 444 stock.dat; (as user nobody) ./bloodbank issue A- 2
stock.dat: Permission denied
$ ./bloodbank
usage: bloodbank find <group> | issue <group> <units> | name-scanf | name-fgets | name-getline
$ ./bloodbank find O+ extra
usage: bloodbank find <group> | issue <group> <units> | name-scanf | name-fgets | name-getline
$ ./bloodbank find Z9
invalid blood group 'Z9'
$ ./bloodbank issue A- 2 && ./bloodbank find A-
issued 2 units of A-, 4 left
A-: 4 units
$ echo $LONGNAME | valgrind --leak-check=full ./bloodbank name-getline
Enter donor name: stored name = ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789ABCD (length 40, buffer 120 bytes)
==641==     in use at exit: 0 bytes in 0 blocks
==641==   total heap usage: 3 allocs, 3 frees, 8,312 bytes allocated
==641== All heap blocks were freed -- no leaks are possible
==641== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Analysis / Discussion

scanf("%s") with a 40-character name into char name[8]: the normal build aborted with '*** stack smashing detected ***' (exit status 134) because the overflow destroyed the stack canary; AddressSanitizer reported 'stack-buffer-overflow ... WRITE of size 41' at bloodbank.c line 34. Without such protection the overflow could overwrite the return address, giving a crash or a security hole. This is the memory error that was found.

Fix: replace scanf("%s") by fgets(name, sizeof name, stdin) (or by scanf with a width, "%7s"). With fgets the name was safely truncated to 'ABCDEFG' (7 characters + '\0'); the remaining characters stay in the input buffer and should be discarded or handled before the next read. After the fix AddressSanitizer reports no error.

Files: a missing data file printed 'missing.dat: No such file or directory' and the program returned 1 instead of crashing; a read-only stock.dat opened for update (run as user nobody) printed 'stock.dat: Permission denied'. Both messages come from perror() and the program stops cleanly.

Arguments: no argument, too many arguments (find O+ extra) print the usage line and return 2; an invalid blood group prints "invalid blood group 'Z9'" and returns 2; a valid command (issue A- 2, find A-) works and updates the file in place.

valgrind on the fixed getline version: 3 allocs, 3 frees, 'All heap blocks were freed', ERROR SUMMARY: 0 errors, so no memory errors remain.

Alternative — getline(): the buffer is allocated with exactly the size needed (120 bytes were allocated for the 40-character line, so no truncation occurs). Reading a line of length L takes O(L) time and uses O(L) heap space (more than the fixed array but only as much as needed). The caller owns the buffer and must free() it, even when getline fails, otherwise there is a leak; fgets needs no free but loses the characters that do not fit.

Result

Task B2 completed. The overflow with scanf was reproduced and explained, the program was corrected with fgets / getline, missing and read-only files and bad arguments are reported without crashing, and valgrind / AddressSanitizer show no remaining memory errors.

TASK B3 — Text File vs Binary File (8 Marks)

Task / Problem Statement

B3.1: write and read back 10 donor records in a text file and a binary file and check they are identical. B3.2: measure the file size and the time to write and read 1,000, 10,000 and 100,000 donors in both formats. B3.3: measure the time to update the 50,000th record in place in the binary file (fseek) and by rewriting the text file. B3.4: explain the differences, including portability, and recommend a format for each kind of data. Complete TODO 1 of update_units(), explain why the update of record 9 fails and why the file is not damaged. Alternative: write all stock records with one fwrite call.

Aim

To compare text and binary files for correctness, size, speed and in-place update, and to make an engineering recommendation.

Concept / Theory

Text I/O (fprintf, fgets, sscanf) converts between characters and values, which costs time and makes records variable length. Binary I/O (fwrite, fread) copies the memory image of a struct directly; fixed-size records allow fseek(index * sizeof(record)) for O(1) access. A binary file depends on the machine: structure padding, the size of int and long, and byte order (endianness) can differ between computers, so it is not automatically portable. Timing was measured with clock_gettime(CLOCK_MONOTONIC) on a warm file cache.

Procedure / Algorithm

1. Complete update_units(): fseek to index*sizeof(Stock), fread (fail if it returns 0), change units, fseek back one record, fwrite, return 1; return 0 if any step fails.
2. B3.1: write 10 donors to d10.txt and d10.bin, read them back and compare every field (text) and memcmp (binary).
3. B3.2: for n = 1,000, 10,000, 100,000 write and read both files and record sizes and times.
4. B3.3: update record 50,000 in the binary file with fseek, and in the text file by reading all lines, changing one, and rewriting everything.
5. Alternative: write 1,000,000 stock records one by one and with a single fwrite and compare calls, time and memory.
6. Analyse the results and write the recommendation.

Program / Source Code

B3 — Completed update_units (TODO 1) — b3_todo.c

```
#include <stdio.h>

typedef struct { int group_code; int units; } Stock; /* fixed-size binary record */

/* Overwrites the units of record number index directly in the binary file. */
int update_units(FILE *fp, int index, int units)
{
    Stock s;
    if (fseek(fp, (long)index * (long)sizeof(Stock), SEEK_SET) != 0) return 0;   /* TODO 1 */
    if (fread(&s, sizeof(Stock), 1, fp) != 1) return 0;     /* record does not exist -> fail, file untouched */
    s.units = units;
    if (fseek(fp, -(long)sizeof(Stock), SEEK_CUR) != 0) return 0;
    if (fwrite(&s, sizeof(Stock), 1, fp) != 1) return 0;
    fflush(fp);
    return 1;
}

int main(void)
{
    FILE *fp = tmpfile();
    if (fp == NULL) { perror("tmpfile"); return 1; }

    Stock initial[5] = {{1, 40}, {2, 25}, {3, 12}, {4, 8}, {5, 30}};
    fwrite(initial, sizeof(Stock), 5, fp);

    printf("update record 3: %s\n", update_units(fp, 3, 20) ? "done" : "failed");
    printf("update record 9: %s\n", update_units(fp, 9, 20) ? "done" : "failed");

    rewind(fp);
    Stock s;
    while (fread(&s, sizeof s, 1, fp) == 1) printf("%d:%d ", s.group_code, s.units);
    printf("\n");
    fclose(fp);
    return 0;
}
```

B3.1–B3.3 — Benchmark program — b3_bench.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

typedef struct { int id; char name[20]; char group[4]; int age; char phone[12]; char last[11]; } Donor;
typedef struct { int group_code; int units; } Stock;

static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv_sec + t.tv_nsec / 1e9; }
static long fsize(const char *p) { FILE *f = fopen(p, "rb"); fseek(f, 0, SEEK_END); long s = ftell(f); fclose(f); return s; }

static void make(Donor *d, int i)
{
    static const char *G[8] = {"O+","O-","A+","A-","B+","B-","AB+","AB-"};
    memset(&d[i], 0, sizeof(Donor));
    d[i].id = 100000 + i;
    snprintf(d[i].name, sizeof d[i].name, "Donor%d", i);
    strcpy(d[i].group, G[i % 8]);
    d[i].age = 18 + i % 48;
    snprintf(d[i].phone, sizeof d[i].phone, "98%09d", i % 1000000000);
    snprintf(d[i].last, sizeof d[i].last, "2026-%02d-%02d", 1 + i % 12, 1 + i % 28);
}

static void write_text(const char *p, const Donor *d, int n)
{
    FILE *f = fopen(p, "w");
    for (int i = 0; i < n; i++) fprintf(f, "%d,%s,%s,%d,%s,%s\n", d[i].id, d[i].name, d[i].group, d[i].age, d[i].phone,
        d[i].last);
    fclose(f);
}
static int read_text(const char *p, Donor *d)
{
    FILE *f = fopen(p, "r"); char line[128]; int n = 0;
    while (fgets(line, sizeof line, f)) {
        Donor *x = &d[n]; memset(x, 0, sizeof *x);
        sscanf(line, "%d,%19[^,],%3[^,],%d,%11[^,],%10s", &x->id, x->name, x->group, &x->age, x->phone, x->last);
        n++;
    }
    fclose(f); return n;
}
static void write_bin(const char *p, const Donor *d, int n)
{ FILE *f = fopen(p, "wb"); for (int i = 0; i < n; i++) fwrite(&d[i], sizeof(Donor), 1, f); fclose(f); }
static int read_bin(const char *p, Donor *d)
{ FILE *f = fopen(p, "rb"); int n = 0; while (fread(&d[n], sizeof(Donor), 1, f) == 1) n++; fclose(f); return n; }

int main(void)
{
    printf("sizeof(Donor) = %zu bytes\n\n", sizeof(Donor));

    /* B3.1 correctness on 10 records */
    Donor a[10], t[10], b[10];
    for (int i = 0; i < 10; i++) make(a, i);
    write_text("d10.txt", a, 10); write_bin("d10.bin", a, 10);
    int nt = read_text("d10.txt", t), nb = read_bin("d10.bin", b);
    int okt = 1, okb = 1;
    for (int i = 0; i < 10; i++) {
        if (t[i].id != a[i].id || strcmp(t[i].name, a[i].name) || strcmp(t[i].group, a[i].group) || t[i].age != a[i].age ||
        strcmp(t[i].phone, a[i].phone) || strcmp(t[i].last, a[i].last)) okt = 0;
        if (memcmp(&a[i], &b[i], sizeof(Donor)) != 0) okb = 0;
    }
    printf("B3.1  text : read back %d records, identical = %s\n", nt, okt ? "YES" : "NO");
    printf("      binary: read back %d records, identical = %s\n\n", nb, okb ? "YES" : "NO");

    /* B3.2 size and time */
    printf("B3.2  %-8s | %-9s %-10s %-9s | %-9s %-10s %-9s\n", "donors", "text KB", "write ms", "read ms", "bin KB", "write
        ms", "read ms");
    int sizes[] = {1000, 10000, 100000};
    for (int s = 0; s < 3; s++) {
        int n = sizes[s];
        Donor *d = malloc((size_t)n * sizeof(Donor)), *r = malloc((size_t)n * sizeof(Donor));
        for (int i = 0; i < n; i++) make(d, i);
        double t0 = now(); write_text("t.txt", d, n); double tw = now() - t0;
        t0 = now(); read_text("t.txt", r); double tr = now() - t0;
        t0 = now(); write_bin("t.bin", d, n); double bw = now() - t0;
        t0 = now(); read_bin("t.bin", r); double br = now() - t0;
        printf("      %-8d | %-9ld %-10.2f %-9.2f | %-9ld %-10.2f %-9.2f\n", n, fsize("t.txt") / 1024, tw * 1e3, tr * 1e3,
        fsize("t.bin") / 1024, bw * 1e3, br * 1e3);
        free(d); free(r);
    }

    /* B3.3 update the 50,000th record (100,000-donor files already on disk) */
    int n = 100000; int target = 49999;           /* record number 50,000 (0-based 49999) */
    double t0 = now();
    FILE *f = fopen("t.bin", "r+b"); Donor x;
    fseek(f, (long)target * sizeof(Donor), SEEK_SET); fread(&x, sizeof x, 1, f);
    x.age = 99;
```

```
    fseek(f, -(long)sizeof(Donor), SEEK_CUR); fwrite(&x, sizeof x, 1, f); fclose(f);
    double tb = now() - t0;

    t0 = now();
    Donor *all = malloc((size_t)n * sizeof(Donor));
    int cnt = read_text("t.txt", all);
    all[target].age = 99;
    write_text("t.txt", all, cnt);
    double tt = now() - t0; free(all);
    printf("\nB3.3  update record 50,000 of 100,000\n");
    printf("      binary (fseek)       = %.4f ms\n      text (rewrite all)   = %.2f ms  (%.0fx slower)\n", tb * 1e3, tt * 1e3,
        tt / tb);

    /* Alternative: one fwrite for the whole stock array vs one fwrite per record */
    int m = 1000000; Stock *st = malloc((size_t)m * sizeof(Stock));
    for (int i = 0; i < m; i++) { st[i].group_code = i % 8; st[i].units = i; }
    t0 = now(); f = fopen("s1.bin", "wb"); for (int i = 0; i < m; i++) fwrite(&st[i], sizeof(Stock), 1, f); fclose(f); double
        per = now() - t0;
    t0 = now(); f = fopen("s2.bin", "wb"); fwrite(st, sizeof(Stock), m, f); fclose(f); double one = now() - t0;
    printf("\nAlternative: %d stock records\n", m);
    printf("      per-record fwrite = %d calls, %.2f ms\n", m, per * 1e3);
    printf("      single fwrite     = 1 call,  %.2f ms, array in RAM = %.1f MB\n", one * 1e3, m * sizeof(Stock) / 1048576.0);
    free(st);
    return 0;
}
```

Output Screenshot

```
Terminal — gcc / bash

$ ./b3_todo
update record 3: done
update record 9: failed
1:40 2:25 3:12 4:20 5:30
```

```
Terminal — gcc / bash

$ ./b3_bench
sizeof(Donor) = 56 bytes

B3.1  text : read back 10 records, identical = YES
      binary: read back 10 records, identical = YES

B3.2  donors   | text KB   write ms   read ms   | bin KB    write ms   read ms
      1000     | 44        2.09       0.26      | 54        2.22       0.02
      10000    | 450       1.53       2.29      | 546       0.59       0.19
      100000   | 4603      13.96      22.66     | 5468      5.25       2.48

B3.3  update record 50,000 of 100,000
      binary (fseek)       = 0.0113 ms
      text (rewrite all)   = 36.20 ms  (3210x slower)

Alternative: 1000000 stock records
      per-record fwrite = 1000000 calls, 15.00 ms
      single fwrite     = 1 call,  8.75 ms, array in RAM = 7.6 MB
```

Analysis / Discussion

TODO output matches the expected output: 'update record 3: done', 'update record 9: failed' and 1:40 2:25 3:12 4:20 5:30. Record 9 fails because the file only holds 5 records: fseek to offset 72 succeeds (seeking beyond the end is allowed) but fread returns 0 because there is nothing to read, so the function returns 0 before reaching fwrite. No byte is written, so the file is not damaged.

B3.1: both formats returned the 10 records identical to the originals (text compared field by field, binary with memcmp).

B3.2 (measured): 1,000 donors: text 44 KB, write 2.09 ms, read 0.26 ms; binary 54 KB, write 2.22 ms, read 0.02 ms (the 1,000-donor write times are dominated by file creation and are noisy). 10,000 donors: text 450 KB, 1.53 ms / 2.29 ms; binary 546 KB, 0.59 ms / 0.19 ms. 100,000 donors: text 4,603 KB, 13.96 ms / 22.66 ms; binary 5,468 KB, 5.25 ms / 2.48 ms. At the larger sizes binary was about 2.5-3 times faster to write and 9-13 times faster to read because no conversion is needed (exact times vary a little from run to run); in this test the binary file was about 19% larger because the fixed 56-byte record keeps padding and unused characters of every field.

B3.3 (measured): updating record 50,000 of 100,000 took 0.011 ms with fseek (binary) and 36.20 ms when the whole text file was rewritten, about 3,000 times slower. The binary update is O(1), the text rewrite is O(n).

B3.4 Trade-off: binary is much faster and allows direct updates, but it is not portable (padding, int size and byte order can differ between computers or compilers), cannot be read in a text editor and breaks if the struct changes. Text is slower and larger to update but portable, readable, easy to debug and to exchange. Recommendation: keep the donor master data (variable fields, exchanged between hospitals, rarely updated) in a text / CSV file; keep the stock table (small, fixed records, updated after every donation) in a binary file, with fixed-width types such as int32_t and documented byte order if it must move between machines.

Alternative — one fwrite for the whole stock array: 1,000,000 records need 1,000,000 fwrite calls when written one by one (15.00 ms) but a single call writes them all (8.75 ms, about 1.7 times faster) because the library overhead per call disappears; the price is that all 1,000,000 records (7.6 MB) must be held in memory before writing. The number of library calls drops from n to 1 and the time is still O(n) in the size of data, but with a much smaller constant.

Result

Task B3 completed. update_units works and fails safely, both formats were verified identical on 10 records, measured at three data sizes, and the in-place update was about 3,000 times faster than rewriting a text file. Learned: pick the format by access pattern and portability needs, not by habit.

TASK B4 — Final Engineering Decision (5 Marks)

Task / Problem Statement

The district now connects 25 hospitals with 100 times the present number of donors, and the blood bank wants a nightly report of donors eligible to donate in the next week. B4.1: implement the report with the growable donor list loaded from the file and measure memory and time. B4.2: show that the program frees all memory. B4.3: decide between keeping all donors in memory, reading the file in batches, or a separate index file, justified by the measurements. Complete TODO 1 of add_donor and explain why realloc's result is stored in a temporary pointer. Alternative: a sorted index file searched with binary search and fseek.

Aim

To implement the nightly eligibility report with a growable list, to verify memory safety, and to take an evidence-based production decision.

Concept / Theory

A donor is eligible in the next week if at least 90 days have passed since the last donation on the day of the report + 7 days. The list grows by doubling so the average cost of adding a donor is O(1). Memory use is measured with getrusage (peak resident set size) and the heap size is capacity * sizeof(Donor). valgrind and AddressSanitizer check that every block is freed and that no invalid access happens. An index file holds fixed-size (id, offset) records sorted by id, so a donor can be found with binary search using fseek: O(log n) file reads and O(1) memory.

Procedure / Algorithm

1. Complete TODO 1: new_cap = 2 if capacity is 0 else double; p = realloc(old, new_cap*sizeof(Donor)); if p is NULL return 0; otherwise data = p and capacity = new_cap.
2. Generate donor files of 5,000 (present size), 20,000 and 500,000 (100 times) donors (id, group, days since last donation).
3. Load the file with add_donor, count donors with days_since + 7 >= 90 per blood group and print the report.
4. Measure the heap size, peak memory and times for load and report at 5,000 and 500,000 donors.
5. Run valgrind (20,000 donors) and AddressSanitizer + UndefinedBehaviourSanitizer (500,000 donors).
6. Build an index file and compare 10,000 random lookups in memory and through the index file; then decide.

Program / Source Code

B4 — Completed add_donor (TODO 1) — b4_todo.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { int id; char group[4]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;

/* Appends a donor, doubling the capacity when the list is full.
   Returns 1 on success, 0 if memory could not be allocated. */
int add_donor(DonorList *list, int id, const char *group)
{
    if (list->size == list->capacity) {
        int new_cap = (list->capacity == 0) ? 2 : list->capacity * 2;       /* TODO 1 */
        Donor *p = realloc(list->data, (size_t)new_cap * sizeof(Donor));    /* temporary pointer */
        if (p == NULL) return 0;          /* old block is still valid in list->data */
        list->data = p;
        list->capacity = new_cap;
    }
    list->data[list->size].id = id;
    strncpy(list->data[list->size].group, group, 3);
    list->data[list->size].group[3] = '\0';
    list->size++;
    return 1;
}

int main(void)
{
    DonorList list = {NULL, 0, 0};
    const char *groups[] = {"O+", "A+", "B-", "AB+", "O-"};
    for (int i = 0; i < 5; i++) {
        add_donor(&list, 7001 + i, groups[i]);
        printf("added %d (%s): size = %d, capacity = %d\n",
               list.data[i].id, list.data[i].group, list.size, list.capacity);
    }
    free(list.data);
    return 0;
}
```

B4.1 — Nightly report program (Donor extended with days_since) — b4_report.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/resource.h>

typedef struct { int id; char group[4]; int days_since; } Donor;   /* Donor + last-donation field */
typedef struct { Donor *data; int size, capacity; } DonorList;
typedef struct { int id; long offset; } IndexRec;

int add_donor(DonorList *list, int id, const char *group, int days)
{
    if (list->size == list->capacity) {
        int new_cap = (list->capacity == 0) ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data, (size_t)new_cap * sizeof(Donor));
        if (p == NULL) return 0;
        list->data = p; list->capacity = new_cap;
    }
    Donor *d = &list->data[list->size];
    d->id = id; strncpy(d->group, group, 3); d->group[3] = '\0'; d->days_since = days;
    list->size++;
    return 1;
}

static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv_sec + t.tv_nsec / 1e9; }

int main(int argc, char *argv[])
{
    if (argc < 2) { fprintf(stderr, "usage: b4_report <donor-file> [index]\n"); return 2; }
    FILE *fp = fopen(argv[1], "r");
    if (fp == NULL) { perror(argv[1]); return 1; }

    DonorList list = {NULL, 0, 0};
    char line[64]; int id, days; char grp[8];
    double t0 = now();
    while (fgets(line, sizeof line, fp)) {
        if (sscanf(line, "%d,%7[^,],%d", &id, grp, &days) != 3) continue;
        if (!add_donor(&list, id, grp, days)) { fprintf(stderr, "out of memory\n"); free(list.data); fclose(fp); return 1; }
    }
    fclose(fp);
    double t_load = now() - t0;

    /* nightly report: donors who will be eligible (>= 90 days since last donation) within the next 7 days */
    static const char *G[8] = {"O+","O-","A+","A-","B+","B-","AB+","AB-"};
    long eligible[8] = {0}; long total = 0;
    t0 = now();
    for (int i = 0; i < list.size; i++)
        if (list.data[i].days_since + 7 >= 90)
            for (int g = 0; g < 8; g++) if (strcmp(list.data[i].group, G[g]) == 0) { eligible[g]++; total++; }
    double t_rep = now() - t0;

    struct rusage ru; getrusage(RUSAGE_SELF, &ru);
    printf("donors loaded      : %d (capacity %d)\n", list.size, list.capacity);
    printf("heap used by list  : %.2f MB (%zu bytes/donor)\n", list.capacity * sizeof(Donor) / 1048576.0, sizeof(Donor));
    printf("peak memory (RSS)  : %.2f MB\n", ru.ru_maxrss / 1024.0);
    printf("load time          : %.1f ms | report time: %.1f ms\n", t_load * 1e3, t_rep * 1e3);
    printf("eligible next week : %ld donors  [", total);
    for (int g = 0; g < 8; g++) printf("%s=%ld%s", G[g], eligible[g], g < 7 ? " " : "");
    printf("]\n");

    if (argc == 3 && strcmp(argv[2], "index") == 0) {      /* compare 10,000 lookups: RAM binary search vs index file + fseek */
        FILE *ix = fopen("donors.idx", "wb");
        for (int i = 0; i < list.size; i++) { IndexRec r = {list.data[i].id, (long)i * (long)sizeof(Donor)}; fwrite(&r, sizeof
        r, 1, ix); }
        fclose(ix);
        ix = fopen("donors.idx", "rb");
        int L = 10000; long found_m = 0, found_f = 0; srand(1);
        int *keys = malloc(L * sizeof(int));
        for (int k = 0; k < L; k++) keys[k] = list.data[rand() % list.size].id;
        t0 = now();
        for (int k = 0; k < L; k++) { int lo = 0, hi = list.size - 1; while (lo <= hi) { int m = (lo + hi) / 2; if
        (list.data[m].id == keys[k]) { found_m++; break; } if (list.data[m].id < keys[k]) lo = m + 1; else hi = m - 1; } }
        double t_mem = now() - t0;
        t0 = now();
        for (int k = 0; k < L; k++) { long lo = 0, hi = list.size - 1; while (lo <= hi) { long m = (lo + hi) / 2; IndexRec r;
        fseek(ix, m * (long)sizeof r, SEEK_SET); fread(&r, sizeof r, 1, ix); if (r.id == keys[k]) { found_f++; break; } if
        (r.id < keys[k]) lo = m + 1; else hi = m - 1; } }
        double t_file = now() - t0;
        printf("lookups (%d)       : in memory %.2f ms (%.3f us each) | index file %.1f ms (%.2f us each), found %ld/%ld\n",
               L, t_mem * 1e3, t_mem / L * 1e6, t_file * 1e3, t_file / L * 1e6, found_m, found_f);
        free(keys); fclose(ix);
    }
    free(list.data);
    return 0;
}
```

Output Screenshot

```
Terminal — gcc / bash

$ ./b4_todo
added 7001 (O+): size = 1, capacity = 2
added 7002 (A+): size = 2, capacity = 2
added 7003 (B-): size = 3, capacity = 4
added 7004 (AB+): size = 4, capacity = 4
added 7005 (O-): size = 5, capacity = 8
```

```
Terminal — gcc / bash

$ ./b4_report donors_5k.txt
donors loaded      : 5000 (capacity 8192)
heap used by list  : 0.09 MB (12 bytes/donor)
peak memory (RSS)  : 1.78 MB
load time          : 0.8 ms | report time: 0.1 ms
eligible next week : 3978 donors  [O+=504 O-=465 A+=523 A-=489 B+=510 B-=471 AB+=543 AB-=473]
$ ./b4_report donors_500k.txt index
donors loaded      : 500000 (capacity 524288)
heap used by list  : 6.00 MB (12 bytes/donor)
peak memory (RSS)  : 7.40 MB
load time          : 65.4 ms | report time: 12.4 ms
eligible next week : 396358 donors  [O+=49915 O-=49713 A+=49354 A-=49460 B+=49560 B-=49665 AB+=49234 A...
lookups (10000)       : in memory 2.06 ms (0.206 us each) | index file 54.0 ms (5.40 us each), found 1...
$ ./b4_report nofile.txt
nofile.txt: No such file or directory
$ valgrind --leak-check=full ./b4_report donors_20k.txt
donors loaded      : 20000 (capacity 32768)
==652==     in use at exit: 0 bytes in 0 blocks
==652==   total heap usage: 18 allocs, 18 frees, 795,072 bytes allocated
==652== All heap blocks were freed -- no leaks are possible
==652== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
$ gcc -fsanitize=address,undefined ... && ./b4_report_asan donors_500k.txt
donors loaded      : 500000 (capacity 524288)
heap used by list  : 6.00 MB (12 bytes/donor)
(no AddressSanitizer / LeakSanitizer report: 0 errors, 0 leaks)
```

Analysis / Discussion

TODO output matches the expected output exactly (capacity 2, 2, 4, 4, 8 for sizes 1 to 5). realloc's result is stored in a temporary pointer p because realloc returns NULL when it fails while the old block is still allocated; writing list->data = realloc(list->data, ...) would overwrite the only pointer to the old block with NULL, causing a memory leak and losing all donors (and a crash on the next access). With the temporary pointer add_donor can return 0 and the program keeps its data.

B4.1 (measured): 5,000 donors use a 0.09 MB list, 1.78 MB peak memory, load 0.8 ms and report 0.1 ms. At 100 times the size, 500,000 donors use a 6.00 MB list (capacity 524,288 x 12 bytes), 7.40 MB peak memory, load 65 ms and report 12 ms, and 396,358 donors become eligible within the week. The list grows 64 times (capacity 8,192 to 524,288) for 100 times the donors because doubling rounds the capacity up, peak memory grows only about 4 times because of the fixed process overhead, and load and report time grow roughly linearly.

B4.2: valgrind on the 20,000-donor run reported 18 allocs, 18 frees, 'All heap blocks were freed -- no leaks are possible' and 0 errors; the AddressSanitizer + UBSan build on 500,000 donors ran without any report. The program frees list.data once on every exit path, including the error paths.

B4.3 Decision: keep all donors in memory. 500,000 donors need only about 6-7 MB, a negligible amount for a server, and the whole nightly run takes about 80 ms. Lookup in memory costs 0.2 microseconds against 5.4 microseconds through the index file (about 26 times slower), and batch reading would add code complexity with no benefit at this size. Reconsider batch reading or an index file only if the data grew by about 100 times again (tens of millions of donors, hundreds of MB) or if memory on the server became limited.

Alternative — sorted index file with fseek + binary search: each lookup needs about log2(500,000) = 19 file reads, O(log n) time per lookup, but only O(1) memory since no donor is loaded, whereas the in-memory list needs no file reads once loaded (my test used binary search in the array, only RAM comparisons) but needs O(n) memory (6 MB). Measured: 10,000 lookups took 2 ms in memory against 54 ms through the index file. The index file is the right choice only when memory is the scarce resource.

Result

Task B4 completed. The report program handles 500,000 donors in about 80 ms using about 7 MB, is free of memory errors, and the decision to keep the donors in memory is supported by the measurements. Learned: measure before choosing an architecture; at this scale the simplest design is also the best.

Test Cases / Validation Table

Record different inputs, expected results, actual results, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1 (A1)	eligible_ptr, O+ list of 6 donors	count 3, youngest id 7004	count = 3, youngest id = 7004	Pass
2 (A1)	eligible_struct, same list	same as pointer version	count = 3, youngest id = 7004	Pass
3 (A2)	digital_root(9875)	2 (29 -> 11 -> 2)	2 (4 calls); formula gives 2	Pass
4 (A2)	power_fast(2, 10)	1024 in floor(log2 10)+2 = 5 calls	1024, 5 calls, depth 5	Pass
5 (A3)	lowest_stock on {40,25,12,8,30,15,5,22}	pointer to element 5 (index 6)	index 6, value 5, *low += 10 -> 15	Pass
6 (A4)	growth copies for n = 100,000	doubling O(n), +10 O(n^2)	131,070 vs 499,950,000 copies	Pass
7 (A5)	in-place update of record 3 to 120 units	only record 3 changes	1:40 2:25 3:12 4:120 ...	Pass
8 (B1)	power_linear(2,30) / power_fast(2,30)	1073741824 in 31 / 6 calls	1073741824 in 31 / 6 calls	Pass
9 (B1)	digital_root(0), (1), (123456789)	0, 1, 9	0, 1, 9	Pass
10 (B1)	call counts n = 10, 100, 1000, 10000	n+1 and floor(log2 n)+2	11/5, 101/8, 1001/11, 10001/15	Pass
11 (B1)	linear recursion, increasing n	crash by stack overflow	OK at 256,000; crash at 512,000 (limit ~261,000)	Pass
12 (B2)	40-character name, scanf("%s") into char[8]	overflow detected	stack smashing detected; ASan stack-buffer-overflow	Pass
13 (B2)	same name with fgets / getline	no overflow	ABCDEFG (truncated) / full 40 chars, 0 errors	Pass
14 (B2)	missing file; read-only file	perror message, no crash	No such file or directory; Permission denied	Pass
15 (B2)	no args; find O+ extra; find Z9	usage / error, exit 2	usage; usage; invalid blood group 'Z9'	Pass
16 (B3)	10 donors in text and binary file	read-back identical	identical = YES (both)	Pass
17 (B3)	update_units record 3 and record 9 (5 records)	done / failed, file unchanged	done / failed, 1:40 2:25 3:12 4:20 5:30	Pass
18 (B3)	update record 50,000 of 100,000	binary much faster	0.011 ms vs 36.20 ms (text)	Pass
19 (B4)	add_donor, 5 donors	capacity 2, 2, 4, 4, 8	capacity 2, 2, 4, 4, 8	Pass
20 (B4)	report on 500,000 donors; valgrind	report ok, no leaks	396,358 eligible, 7.4 MB, 0 errors, no leaks	Pass

Final Submission Checklist

☑ Student and course details completed

☑ Task statement, aim, theory, and algorithm included

☑ Program code is readable and runs successfully

☑ Output screenshot included

☑ Analysis and result written in own words

☑ Test cases completed where applicable

☐ GitHub repository link added if required